

them, demonstrating that *diffu*-GRPO achieves stronger overall performance than SFT alone. Both LLaDA+*diffu*-GRPO and d1-LLaDA demonstrate consistent improvements over their respective starting points. Specifically, LLaDA+*diffu*-GRPO outperforms the base LLaDA-8B-Instruct model across all setups, and d1-LLaDA surpasses LLaDA+SFT in every case. This indicates that *diffu*-GRPO provides reliable performance gains, regardless of the initialization—whether from a pretrained model or an SFT-adapted checkpoint.

d1 recipe yields the highest gains. SFT, followed by *diffu*-GRPO—resulting in d1-LLaDA—yields additional gains, beyond either method individually. This combined approach outperforms pure *diffu*-GRPO in 11 out of 12 setups, indicating a synergistic effect between the two training stages. Notably, while d1-LLaDA shows consistent improvements across all benchmarks, the magnitude varies by task: we observe modest improvements on GSM8K (3.9%) and MATH500 (4.0%), but significantly larger gains on Countdown (26.2%) and Sudoku (10.0%). We hypothesize this discrepancy stems from the base model’s saturation on mathematical tasks, with less room for improvement as compared to planning benchmarks that involve structured constraint satisfaction patterns.

Training a unified model across tasks retains strong performance. We train a single *diffu*-GRPO (and d1) model on the *combined* GSM8K, MATH500, Countdown, and Sudoku datasets. To ensure balanced training, we subsample the data so that each task has the same number of training examples. Even with subsampling, Table 2 shows that *diffu*-GRPO scales well to multi-task settings without sacrificing accuracy compared to the per-task *diffu*-GRPO results in Table 1.

Scaling *diffu*-GRPO to coding domains. We also evaluate *diffu*-GRPO on coding tasks, where we train a model on the KodCode-Light-RL-10K dataset [45], which contains general coding tasks with solutions verified by synthetic unit tests. The *diffu*-GRPO results are shown in Table 3. We find that *diffu*-GRPO consistently improves performance, regardless of the initialization point. Interestingly, our findings suggest that s1k is not suitable for coding, since it **lacks datapoints with code**. Exploration into finding the optimal SFT dataset is left for future works.

***diffu*-GRPO improves reasoning beyond training sequence length.** Although our *diffu*-GRPO training uses fixed sequence length of 256 for online generations, we observe performance gains at other generation sequence lengths as well. The improvements at 128 and 512 sequence lengths suggest that the model has learned more general reasoning strategies rather than overfitting to a specific length. This is further supported by the effective token usage data, presented in Figure 4, which shows no truncation at 128 tokens and increased token utilization at 512.

4.3 Discussion

Qualitative results show “aha moments” in SFT and d1-LLaDA generations. While the performance for generation sequence length 128 and 256 increases with SFT, *diffu*-GRPO and d1 as compared to LLaDA-8B-Instruct, qualitatively, we do not observe significant differences in the generated reasoning traces. However, at sequence length 512, we begin observing “aha moments” in the SFT and d1-LLaDA models, which demonstrates self-correction and backtracking behaviors. We show these in Appendix E. For the same questions from GSM8k, we show generations of each model, with the variants using SFT showing self-verifications and self-corrections to the right answer. Our intuition is that the model has instilled behaviors such as verification of intermediate results and backtracking from the reasoning traces of s1k during the SFT stage.

Table 2: **Unified Model Performance Across Reasoning Tasks:** For *diffu*-GRPO and d1-LLaDA variants, a single model was trained on the combined dataset of GSM8K, MATH500, Countdown, and Sudoku. **Green** and **blue values** indicate the best and second-best performance.

Model / Seq Len	GSM8K		MATH500		Countdown		Sudoku	
	128	256	128	256	128	256	128	256
LLaDA-8B-Instruct	68.7	76.7	26.0	32.4	20.7	19.5	11.7	6.7
+ SFT (s1k)	66.5	78.8	26.2	32.6	20.3	14.5	16.5	8.5
+ combined <i>diffu</i> -GRPO	72.4	78.2	30.2	36.6	27.7	19.5	22.9	15.7
combined d1-LLaDA	75.1	81.1	29.8	35.4	30.1	32.8	21.9	15.4

Table 3: **Effectiveness of *diffu*-GRPO on Coding Benchmarks:** Evaluated with and without *diffu*-GRPO on HumanEval and MBPP. *diffu*-GRPO consistently improves over initialization checkpoint on coding tasks.

Model / Seq Len	HumanEval			MBPP		
	128	256	512	128	256	512
LLaDA-8B-Instruct	27.4	35.3	37.8	36.2	41.2	40.4
+ <i>diffu</i> -GRPO	29.3	39.0	34.8	42.0	45.5	41.6
Δ (<i>diffu</i> -GRPO gain)	+1.9	+3.7	-3.0	+5.8	+4.3	+1.2
LLaDA-8B-Instruct + SFT (s1k)	21.3	32.3	32.9	40.1	39.7	41.2
+ <i>diffu</i> -GRPO	31.1	32.9	37.8	40.5	44.7	42.8
Δ (<i>diffu</i> -GRPO gain)	+9.8	+0.6	+4.9	+0.4	+5.0	+1.6

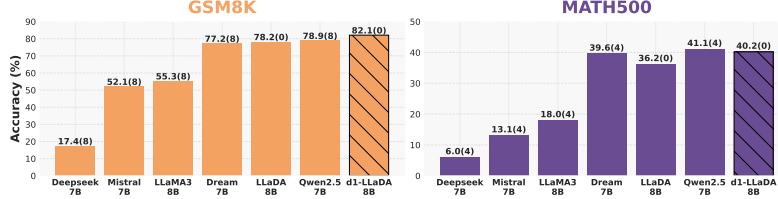


Figure 3: **Comparison with state-of-the-art dLLMs and AR LLMs of similar size:** d1-LLaDA achieves the highest GSM8K score and the second-highest MATH500 score. LLaDA results are from our evaluation using 0-shot. Scores for other models are from Dream [48], using 8-shot prompts for GSM8K and 4-shot for MATH. Note that here we report d1-LLaDA with task-specific RL training.

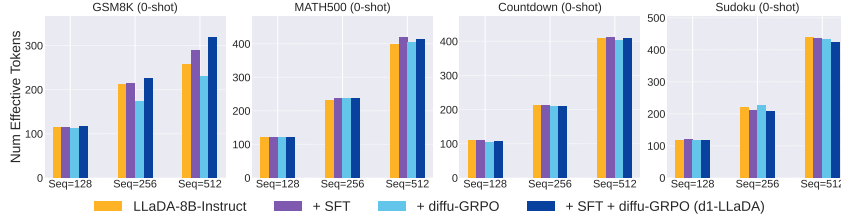


Figure 4: **Effective Token Usage:** As we increase the evaluation generation length, the number of effective tokens (average number of non-padding, non-EOS tokens per generation across tasks) grows and remains comparable for all the methods on MATH500, Countdown and Sudoku tasks.

Sequential scaling with increasing generation sequence lengths. LLaDA-8B-Instruct, SFT, *diffu*-GRPO and d1-LLaDA demonstrate improved performance with increasing sequence lengths for GSM8k and MATH500, with larger jumps observed from 128 to 256 ($\sim 7.1\%$), than from 256 to 512 ($\sim 2.5\%$). Qualitative examples in [Appendix E](#) show more sophisticated reasoning traces emerge with 512-token generation lengths. These findings align with previous research showing that increasing test-time compute through longer reasoning processes leads to improved performance in autoregressive models [28]. However, we notice a mixed scaling trend on Countdown and Sudoku. Performance decreases with increasing sequence lengths for Sudoku across all models. For Countdown, LLaDA-8B-Instruct decreases monotonically with sequence length, while SFT, *diffu*-GRPO and d1-LLaDA peak at 512 sequence length. This likely stems from extensive searching requirements, beyond LLaDA-8B-Instruct’s capabilities. We hypothesize favorable sequential scaling will strengthen with more robust base dLLMs. Unlike AR models like DeepSeek R1 [17], we observe no significant CoT length growth post-RL training, as LLaDA-8B-Instruct was pre-trained on sequences up to 4096 tokens. Further scaling requires larger generation lengths during RL training, currently infeasible due to slow generation speed. Future research should develop efficient inference algorithms for online sampling to scale dLLM RL training.

4.4 Design Choices and Ablations for *diffu*-GRPO

Random Masking for Likelihood Estimation Offers Implicit Regularization Our randomized masking mechanism provides significant advantages for training masked dLLMs. As shown in Figure 5, random masking consistently outperforms fixed masking across different values of policy optimization updates (μ). While conventional approaches typically limit μ to 2 due to diminishing returns and overfitting risks, our approach enables scaling μ to much higher values (12, or even 24) while maintaining or improving performance, facilitating faster convergence of RL training. Consequently, fewer number of generations are needed, which in turn remarkably reduces the computational cost. The rightmost plot demonstrates the real-world efficiency gains, where models with higher μ values achieve better correctness rewards in significantly lesser wall clock time. This efficiency stems from creating diverse views of the input data during each optimization step, allowing the model to prevent in-batch overfitting and extract more learning signal from each generation.

Effect of Masking Rate on Training Stability and Performance We examine how prompt masking probability p_{mask} influences *diffu*-GRPO training. As shown in Figure 6, lower rates (0.1, 0.3) yield more stable training and better final performance by preserving more context tokens without masking

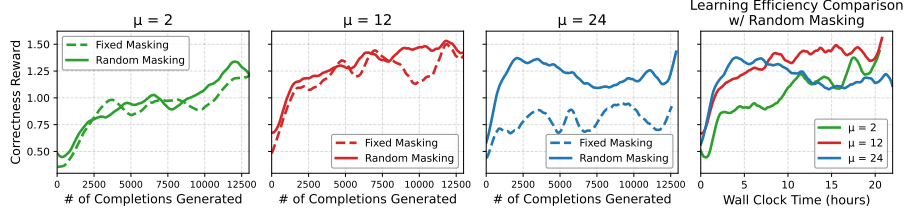


Figure 5: **Comparison of fixed vs. random masking** across different policy optimization update values (μ). The first three figures show GSM8K correctness reward vs. the number of completions generated during RL training with different μ . Random masking consistently outperforms fixed masking. The rightmost panel compares all three μ values with random masking in terms of wall clock time, indicating higher efficiency from higher μ values.

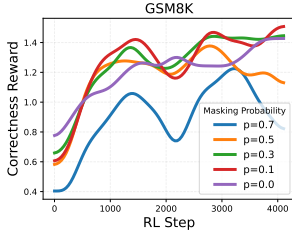


Figure 6: **Ablation of prompt masking probability (p_{mask})** on GSM8K reward trends. Light masking (0.1, 0.3) improves stability and performance over no masking (0.0), suggesting the regularization benefit of random masking as discussed in Sec 3.2. Higher masking rates (0.5, 0.7) introduce instability in later training stages.

them, while higher rates (0.5, 0.7) introduce instability, with 0.7 causing sharp degradation after 3000 steps. Although $p_{\text{mask}} = 0.0$ avoids variability, it underperforms slightly, confirming the regularization effect brought by random masking as discussed in Sec. 3.2. This effect is especially beneficial at large policy iteration counts ($\mu = 12$), as used in this ablation.

5 Related Works

Due to space constraint, we provide a detailed related works discussion in Appendix B.

Diffusion Language Models. Diffusion models, successful in visual domains [40, 19], faced challenges in language due to text’s discrete nature, initially tackled by modeling continuous diffusion on textual latents [5, 16]. Masked diffusion emerged as an effective discrete variant [5, 36, 39, 32, 29], scaled notably in DiffuLLaMA [15], which initialized with pretrained LLaMA weights. Recent works explored chain-of-thought reasoning [47, 46], block-based generation [4], and large-scale competitive performance in LLaDA [30] and Dream [48]. However, reinforcement learning (RL) enhancement remains unexplored; we present the first demonstration using policy gradients for large diffusion language models. **Improving Reasoning Abilities of LLMs through SFT and RL.** Reasoning improvements in LLMs involve supervised finetuning (SFT) with high-quality reasoning datasets [50, 21, 35] or curated reasoning demonstrations [49, 28, 52]. However, RL approaches [9] generalize better, especially with methods like GRPO [17, 38], facilitating advantage estimation without critic models. Advanced reasoning via RL alone was shown by DeepSeek-R1-Zero [17], whose reasoning traces can be used to distill smaller-model, such as OpenThoughts [42] and OpenR1-Math⁴. Prior RL work in discrete diffusion models [51] employed concrete score matching and applied to smaller scale models, whereas our method specifically applies to large masked dLLMs with efficient masking-based policy gradients, integrating both SFT and RL.

6 Conclusion

In this work, we explore scaling reasoning in diffusion LLMs through different recipes. SFT on reasoning datasets improves performance and reveals “Aha moments”. We introduce *diffu*-GRPO, an efficient policy gradient method for dLLMs that consistently outperforms SFT across benchmarks. Combining these approaches, our d1 recipe—a two-stage SFT and *diffu*-GRPO pipeline—delivers the most significant improvements over the baseline. Future work should focus on developing efficient decoding strategies to scale generation length for more effective RL training.

⁴<https://huggingface.co/datasets/open-r1/OpenR1-Math-220k>